GADE7322 · PART 3 · VIDEO PRESENTATION

Presentation cards

12 cards · about 5:50 · say it, show it, then show the code

How to use these cards

Each card has three panels. SAY is the script. SHOW IN GAME is what to do on screen while you say it. SHOW IN CODE gives the exact file, the exact line numbers and the code itself, with a one-line note on what to say about it.

Speak from the key points, not word for word. The brief rewards a confident, natural delivery. Practise three times out loud against the clock.

The brief wants five to six minutes, individual, with audio. These cards run 5:50. Record in one take per card if it helps, then join them.

Line numbers are real. In Visual Studio or Rider press Ctrl+G and type the first line number to jump straight to the code.

Card	Topic	Length	Clock
1	Introduction	0:20	0:00 to 0:20
2	The game: look and feel	0:25	0:20 to 0:45
3	Upgrades: two tiers each	0:40	0:45 to 1:25
4	Shaders	0:35	1:25 to 2:00
5	Visual effects: particles and a post-process	0:35	2:00 to 2:35
6	My procedural feature: Evolving Blight	0:45	2:35 to 3:20
7	How the evolution works	0:30	3:20 to 3:50
8	Plugging it into the game	0:20	3:50 to 4:10
9	More procedural enemies: boids and cellular poison	0:40	4:10 to 4:50
10	Balance: waves react to upgrades	0:25	4:50 to 5:15
11	Challenges and how I solved them	0:20	5:15 to 5:35
12	Conclusion	0:15	5:35 to 5:50

Before you press record (checklist)

[ ] Latest build compiled and the editor closed, then launch the game on the TowerDefense level (not the editor viewport).

[ ] Have these files open in tabs, in this order: DefenderUpgradeComponent.cpp,

> UpgradeVisualComponent.cpp, Enemy.cpp, DangerScreenEffectComponent.cpp, EnemyGenome.h/.cpp, EnemyEvolution.cpp, EnemyEvolutionTests.cpp, WaveManager.cpp, WaveDirector.cpp.
>
> [ ] Open M_CorruptionPulse in the material editor and leave it in a separate window.
>
> [ ] Have the planning document open at Figures 1 to 3.
>
> [ ] Run the automated tests once beforehand so the result is ready to show: Automation RunTests ProceduralGen.EnemyEvolution
>
> [ ] Make sure you have upgrade currency (wallet) in the menu so tier I and tier II can both be bought.
>
> [ ] Close chat apps and notifications. Test your microphone: the brief marks a presentation with no audio as a fail.

Key	Does	Key	Does
B	Open or close the upgrade screen	1 2 3 4	Pick Basic, Archer, Bomb, Strong defender
R	Restart with a new map	P	Pause
G	Show the wave director graph	Tab	Switch strong defender mode
U	Upgrade the tower beam	F5	Quick save
Left click	Place the selected defender on a pad	N	Toggle the NavMesh debug view

CARD 1 OF 12

Introduction

0:20

clock 0:00 to 0:20

SAY (about 44 words, roughly 0:20 at a calm pace)

Hi, I'm Harriet Manda, student number ST10252836. This is our tower defence game, built in Unreal Engine 5 with C++. For Part 3 I'll show the upgrades, two shaders, two visual effects, and my own procedural feature: enemies that evolve from wave to wave.

SHOW IN GAME

1. Have the game window ready on the start screen (StartScreenLvl).
2. Smile and look at the camera while you say the first sentence.
3. Click Start to load the TowerDefense level while you finish speaking.

SHOW IN CODE

No code on this card. Keep the game on screen.

CARD 2 OF 12

The game: look and feel

0:25

clock 0:20 to 0:45

SAY (about 66 words, roughly 0:30 at a calm pace)

Every match builds a new terrain from a seed, with lanes leading to the central tower. I replaced the placeholder shapes with free CC0 models: Kenney's stone tower and turrets, and Quaternius animals for the enemies. The lighting is dimmed and cool with a violet fog for a fantasy-horror mood, glowing crystals line the lanes, and the bottom bar shows the defender cards with their cost.

SHOW IN GAME

1. The match is running: terrain, lanes, the brick tower in the middle, crystals beside the lanes.
2. Point at the bottom defender cards: hotkey, cost, the selected card raised and highlighted.
3. Press F1 to show the developer panel (seed, director, Blight) and press it again to hide it.

If you have time, also point out:

- Models: Kenney Tower Defense Kit and Quaternius animal packs, both CC0 (credited in the group report).
- If a model is missing the code falls back to the old shape, so the game never breaks.

SHOW IN CODE

Source/ProceduralGenGade3B/TDHUD.cpp lines 501 to 503 (Ctrl+G, type 501)

Say: Each card knows if it is selected and affordable, so the bar updates itself from the game state.

```
 501          const bool bSelected = Defaults && Active == Card.Class;
 502          const bool bAfford = Defaults && GameMode->GetResources() >= Defaults->Cost && GameMode-
>CanAffordMeta(Defaults->MetaCost);
 503          const float Dim = bAfford ? 1.0f : 0.45f;
```

SHOW IN CODE

Source/ProceduralGenGade3B/TDGameMode.cpp lines 84 to 105 (Ctrl+G, type 84)

Say: The mood: a dimmer, cool-blue sun and sky light, plus a low violet fog added in code.

```
 84  static void ApplyMoodLighting(UWorld* World)
 85  {
 86      if (!World)
 87      {
 88          return;
 89      }
 90      for (TActorIterator<ADirectionalLight> It(World); It; ++It)
 91      {
 92          if (UDirectionalLightComponent* Sun = Cast<UDirectionalLightComponent>(It-
>GetLightComponent()))
 93          {
 94              Sun->SetLightColor(FLinearColor(0.62f, 0.7f, 1.0f));
 95              Sun->SetIntensity(Sun->Intensity * 0.55f);
 96          }
 97      }
 98      for (TActorIterator<ASkyLight> It(World); It; ++It)
 99      {
100          if (USkyLightComponent* Sky = It->GetLightComponent())
101          {
102              Sky->SetLightColor(FLinearColor(0.5f, 0.55f, 0.85f));
103              Sky->SetIntensity(Sky->Intensity * 0.65f);
104          }
105      }
```

CARD 3 OF 12

Upgrades: two tiers each

0:40

clock 0:45 to 1:25

SAY (about 80 words, roughly 0:36 at a calm pace)

Every defender type and the tower now has two upgrade tiers. I press B to open the upgrade screen and buy tier one for the tower. The same button now offers tier two, which is stronger and costs more. Buying it raises health, damage and fire rate, and the unit changes visually: a gold orb floats above it, and tier two adds a glowing ring. In code, TryActivate works out the next tier, refuses a duplicate purchase and charges once.

SHOW IN GAME

1. Press B to open the Defenders screen. Point at the Tower button.
2. Click the Tower upgrade: the button turns green and reads 'TIER I - CLICK FOR II'.
3. Click it again: it turns gold and reads 'TIER II - MAX'. Close with B.
4. Zoom the camera on the tower, then on a placed defender: show the orb and the ground ring.

If you have time, also point out:

- Tiers reuse the same button, so no UI blueprint had to change.
- Fixed a bug on the way: clicking an already-active upgrade used to charge again for no effect.

SHOW IN CODE

Source/ProceduralGenGade3B/DefenderUpgradeComponent.cpp lines 426 to 431 (Ctrl+G, type 426)

Say: The guard: work out the next tier and refuse a third purchase, so a click on an active upgrade can never charge twice.

```
426     const int32 NextTier = Slot.Tier + 1;
427     if (NextTier > MaxTier || Slot.Phase == EPhase::DisabledAfterWave || Slot.CooldownSecondsRemaining
> 0
428         || !GameMode || GameMode->IsGameOver() || GameMode->IsVictory() || IsPending(Upgrade))
429     {
430         return;
431     }
```

SHOW IN CODE

Source/ProceduralGenGade3B/DefenderUpgradeComponent.cpp lines 465 to 472 (Ctrl+G, type 465)

Say: Tier I activates the upgrade; tier II raises the same units to the stronger stats.

```
465     if (NextTier == 1)
466     {
467         StartActive(Upgrade, Targets);
468     }
469     else
470     {
471         StartTierTwo(Upgrade, Targets);
472     }
```

SHOW IN CODE

Source/ProceduralGenGade3B/DefenderUpgradeComponent.cpp lines 46 to 52 (Ctrl+G, type 46)

Say: Tier II stats live in one table: about 30% stronger for 1.6 to 2 times the price.

```
46      const FUpgradeConfig Tier2Configs[] = {
47          { TEXT("BasicButton"),  165.0f, 5.5f, 22.0f, 400 },
48          { TEXT("ArcherButton"), 150.0f, 7.0f, 20.0f, 110 },
49          { TEXT("PoisonButton"), 175.0f, 5.0f, 28.0f, 30 },
50          { TEXT("StrongButton"), 330.0f, 3.8f, 38.0f, 450 },
51          { TEXT("TowerButton"),  900.0f, 2.4f, 85.0f, 1300 },
52      };
```

SHOW IN CODE

Source/ProceduralGenGade3B/UpgradeVisualComponent.cpp lines 35 to 56 (Ctrl+G, type 35)

Say: Shows the tier on the unit: orb at tier I, orb plus ring at tier II, gold body tint. It never scales the mesh.

```
35  void UUpgradeVisualComponent::SetTier(int32 NewTier)
36  {
37      NewTier = FMath::Clamp(NewTier, 0, 2);
38      if (NewTier == Tier && (NewTier == 0 || OrbMesh))
39      {
40          return;
41      }
42      Tier = NewTier;
43
44      UStaticMeshComponent* Body = GetBodyMesh();
45      if (!Body)
46      {
47          return;
48      }
49
50      if (Tier > 0)
51      {
52          EnsureMeshes(Body);
53      }
54      Layout(Body);
55      ApplyBodyTint(Body);
56  }
```

CARD 4 OF 12

Shaders

0:35

clock 1:25 to 2:00

SAY (about 70 words, roughly 0:32 at a calm pace)

I made two shaders in one material. First, vertex displacement: M_CorruptionPulse pushes every vertex along its normal by a sine wave of time and world position, so the enemy bodies ripple. Here is the graph in the material editor. Second, the same material blends each enemy's base colour with a pulse colour, so the colour breathes. C++ sets the parameters per enemy, so no two mutants ripple the same way.

SHOW IN GAME

1. Switch to the Unreal editor: open Content/Materials/Displacement/M_CorruptionPulse.
2. Point at the chain: Time x PulseSpeed + WorldPosition phase -> Sine -> x PulseAmplitude -> x VertexNormalWS -> World Position Offset.
3. Point at the Lerp into Base Color and the Emissive output.
4. Back to the game: watch enemies walking down a lane, ripple and colour pulse visible.

If you have time, also point out:

- World Position Offset is the material output that moves vertices: it is what makes this a vertex-displacement shader.
- The material was built with the editor's material graph, scripted from Python so it can be rebuilt.

SHOW IN CODE

Source/ProceduralGenGade3B/Enemy.cpp lines 402 to 427 (Ctrl+G, type 402)

Say: Loads the displacement material and writes the type colour and a brighter pulse colour into it.

```
 402          // Prefer the pulsing displacement shader. If it is missing, fall back to whatever the mesh
already uses.
 403          UMaterialInterface* BaseMaterial = BodyMaterialBase.LoadSynchronous();
 404          if (!BaseMaterial)
 405          {
 406              BaseMaterial = MeshComponent->GetMaterial(0);
 407          }
 408          if (!BaseMaterial)
 409          {
 410              return;
 411          }
 412          BodyMaterial = UMaterialInstanceDynamic::Create(BaseMaterial, this);
 413          MeshComponent->SetMaterial(0, BodyMaterial);
 414      }
 415
 416      if (BodyMaterial)
 417      {
 418          BodyMaterial->SetVectorParameterValue(TEXT("Color"), BodyColor);
 419
 420          // The glow colour is a brighter version of the type colour, so a wolf pulses pale blue and a
bear
 421          // pulses warm brown instead of every enemy pulsing the same green.
 422          BodyMaterial->SetVectorParameterValue(TEXT("PulseColor"),
 423              FLinearColor(FMath::Min(1.0f, BodyColor.R * 1.6f + 0.15f),
 424                           FMath::Min(1.0f, BodyColor.G * 1.6f + 0.15f),
 425                           FMath::Min(1.0f, BodyColor.B * 1.6f + 0.15f)));
 426      }
 427  }
```

SHOW IN CODE

Source/ProceduralGenGade3B/Enemy.cpp lines 130 to 136 (Ctrl+G, type 130)

Say: Each genome sets PulseSpeed, PulseAmplitude and EmissiveStrength, so every mutant ripples differently.

```
 130      // ---- Shader. These are the parameters of M_CorruptionPulse, so every genome ripples
differently. ----
 131      if (BodyMaterial)
 132      {
 133          BodyMaterial->SetScalarParameterValue(TEXT("PulseSpeed"), 1.5f + 4.0f * Genome.Swiftness);
 134          BodyMaterial->SetScalarParameterValue(TEXT("PulseAmplitude"), 3.0f + 11.0f * Genome.Pulse);
 135          BodyMaterial->SetScalarParameterValue(TEXT("EmissiveStrength"), 0.3f + 0.9f * Genome.Fury);
 136      }
```

CARD 5 OF 12

Visual effects: particles and a post-process

0:35

clock 2:00 to 2:35

SAY (about 68 words, roughly 0:31 at a calm pace)

There are two custom effects. Belinda built the Niagara particle systems: tower shots, upgrade bursts and enemy spawns and deaths. I built the screen-space effect. It is a post-process material that drains the colour and pulses a red vignette. A component reads the tower's health every frame. Below forty-five percent the effect fades in, and every hit adds a flash. Watch the screen as the tower takes damage.

SHOW IN GAME

1. Let the tower shoot: point out the muzzle flash and projectile trail (Niagara).
2. Buy an upgrade: point out the upgrade burst around the tower.
3. Place no defenders for a moment and let enemies reach the tower.
4. Show the red vignette and grey colour drain growing as tower health falls (about 45% health and below).

If you have time, also point out:

- The post-process material (PP_TowerDanger) is applied through an unbound post-process volume, so it works with any camera.
- Scene Texture -> Desaturation, plus a screen-centre distance vignette and a sine 'heartbeat'.

SHOW IN CODE

Source/ProceduralGenGade3B/DangerScreenEffectComponent.cpp lines 78 to 96 (Ctrl+G, type 78)

Say: Reads tower health, flashes on every drop, and returns the target intensity from 0 to 1.

```
78      const float Percent = Health->GetHealthPercent();
79
80      // A drop in health since last frame means the tower was just hit, so flash.
81      if (Percent < LastHealthPercent - KINDA_SMALL_NUMBER)
82      {
83          HitFlashRemaining = FMath::Min(1.0f, HitFlashRemaining + HitFlash);
84      }
85      LastHealthPercent = Percent;
86      HitFlashRemaining = FMath::Max(0.0f, HitFlashRemaining - HitFlashFadePerSecond * DeltaTime);
87
88      // Danger rises from 0 at the threshold to 1 at zero health. A dead tower is always full strength.
89      float Danger = FMath::Clamp((DangerStartsBelow - Percent) / DangerStartsBelow, 0.0f, 1.0f);
90      if (Health->IsDead())
91      {
92          Danger = 1.0f;
93      }
94
95      return FMath::Clamp(Danger + HitFlashRemaining, 0.0f, 1.0f);
96  }
```

SHOW IN CODE

Source/ProceduralGenGade3B/DangerScreenEffectComponent.cpp lines 107 to 113 (Ctrl+G, type 107)

Say: Eases the picture towards the target and writes it into the material's Intensity parameter.

```
107      const float Target = ComputeTargetIntensity(DeltaTime);
108      CurrentIntensity = FMath::FInterpTo(CurrentIntensity, Target, DeltaTime, ResponseSpeed);
109      if (CurrentIntensity < 0.002f && Target <= 0.0f)
110      {
111          CurrentIntensity = 0.0f; // fully off, so nothing is drawn while the tower is healthy
112      }
113      Material->SetScalarParameterValue(TEXT("Intensity"), CurrentIntensity);
```

CARD 6 OF 12

My procedural feature: Evolving Blight

0:45

clock 2:35 to 3:20

SAY (about 68 words, roughly 0:31 at a calm pace)

My own procedural feature is Evolving Blight. Instead of fixed enemy types, every enemy carries a genome of six genes. Hardiness, Swiftness and Fury share a fixed biomass budget, so a fast enemy has to be weaker elsewhere. Girth, Pulse and Hue only change how it looks. The code clamps every genome to the budget, and each gene maps to a multiplier that equals one at the average.

SHOW IN GAME

1. Hold up the planning document, Figure 1 (the loop) and Figure 2 (biomass budget), for about five seconds each.
2. Back in game: point at enemies with different sizes, colours and ripple speeds on the same lane.
3. Point at the HUD line 'Blight generation N | champion: <name>' after the first wave ends.

If you have time, also point out:

- Named from the two strongest genes, for example 'Moldspore the Swift'.
- Seeded by the map seed, so the same map and play reproduce the same Blight.

SHOW IN CODE

Source/ProceduralGenGade3B/EnemyGenome.h lines 57 to 66 (Ctrl+G, type 57)

Say: What each gene means. Every multiplier is 1.0 at a gene value of 0.5, so an average genome plays like the old enemy.

```
  57      float HealthMultiplier() const { return 0.5f + Hardiness; }
  58      float SpeedMultiplier() const { return 0.6f + 0.8f * Swiftness; }
  59      float DamageMultiplier() const { return 0.5f + Fury; }
  60      float SizeScale() const { return 0.75f + 0.5f * Girth; }
  61
  62      /** Total of the three power genes. This is the number the biomass budget limits. */
  63      float BiomassUsed() const { return Hardiness + Swiftness + Fury; }
  64
  65      /** Overall strength, 1.0 for an even split of the starting budget. Stronger genomes drop more
loot. */
  66      float PowerRating() const { return BiomassUsed() / 1.5f; }
```

SHOW IN CODE

Source/ProceduralGenGade3B/EnemyGenome.cpp lines 12 to 31 (Ctrl+G, type 12)

Say: Keeps genes in range and, if the three power genes exceed the budget, shrinks them together so a tank stays a tank.

```
 12  void FEnemyGenome::ClampToBudget(float Budget)
 13  {
 14      Hardiness = FMath::Clamp(Hardiness, MinGene, MaxGene);
 15      Swiftness = FMath::Clamp(Swiftness, MinGene, MaxGene);
 16      Fury = FMath::Clamp(Fury, MinGene, MaxGene);
 17      Girth = FMath::Clamp(Girth, MinGene, MaxGene);
 18      Pulse = FMath::Clamp(Pulse, MinGene, MaxGene);
 19      Hue = FMath::Clamp(Hue, 0.0f, 1.0f);
 20
 21      // Over budget: shrink all three power genes by the same factor, so the shape of the build stays
the same
 22      // (a tank stays a tank) but it fits the budget.
 23      const float Used = BiomassUsed();
 24      if (Used > Budget && Used > KINDA_SMALL_NUMBER)
 25      {
 26          const float Scale = Budget / Used;
 27          Hardiness = FMath::Max(MinGene, Hardiness * Scale);
 28          Swiftness = FMath::Max(MinGene, Swiftness * Scale);
 29          Fury = FMath::Max(MinGene, Fury * Scale);
 30      }
 31  }
```

CARD 7 OF 12

How the evolution works

0:30

clock 3:20 to 3:50

SAY (about 73 words, roughly 0:33 at a calm pace)

Evolution lives in one class, UEnemyEvolution. When a wave ends, EndWave sorts the enemies by fitness, which is how far they travelled plus a bonus for reaching the tower, and keeps the best forty percent as parents. The champion is copied unchanged. MakeChild does uniform crossover and mutates each gene with a twenty-five percent chance. I tested it automatically: in the selection test, average swiftness rises from 0.46 to 0.94 over eight generations.

SHOW IN GAME

1. Show the Output Log line: 'EnemyEvolution: generation 0 ended ... Champion ...' (filter the log for EnemyEvolution).
2. Show the Automation window or terminal with the four ProceduralGen.EnemyEvolution tests passing.
3. Optional: show the lineage figure (Figure 3) in the planning document.

If you have time, also point out:

- Four tests: budget respected, same seed same Blight, evolution climbs, champion survives unchanged.
- Run them with: Automation RunTests ProceduralGen.EnemyEvolution

SHOW IN CODE

Source/ProceduralGenGade3B/EnemyEvolution.cpp lines 58 to 88 (Ctrl+G, type 58)

Say: Uniform crossover from two parents, then mutation, then the biomass clamp and a generated name.

```
58      // Uniform crossover: every gene comes from one parent or the other with equal chance.
59      const FEnemyGenome& A = Parents[Stream.RandRange(0, Parents.Num() - 1)];
60      const FEnemyGenome& B = Parents[Stream.RandRange(0, Parents.Num() - 1)];
61      auto Pick = [this](float FromA, float FromB) { return Stream.FRand() < 0.5f ? FromA : FromB; };
62
63      FEnemyGenome Child;
64      Child.Hardiness = Pick(A.Hardiness, B.Hardiness);
65      Child.Swiftness = Pick(A.Swiftness, B.Swiftness);
66      Child.Fury = Pick(A.Fury, B.Fury);
67      Child.Girth = Pick(A.Girth, B.Girth);
68      Child.Pulse = Pick(A.Pulse, B.Pulse);
69      Child.Hue = Pick(A.Hue, B.Hue);
70
71      // Mutation: each gene has a chance to shift by a small random amount.
72      auto Mutate = [this](float& Gene)
73      {
74          if (Stream.FRand() < MutationChance)
75          {
76              Gene += Stream.FRandRange(-1.0f, 1.0f) * MutationStrength;
77          }
78      };
79      Mutate(Child.Hardiness);
80      Mutate(Child.Swiftness);
81      Mutate(Child.Fury);
82      Mutate(Child.Girth);
83      Mutate(Child.Pulse);
84      Mutate(Child.Hue);
85
86      Child.Generation = Generation;
87      Child.ClampToBudget(GetBiomassBudget());
88      Child.Name = FEnemyGenome::BuildName(Child, Stream);
```

SHOW IN CODE

Source/ProceduralGenGade3B/EnemyEvolution.cpp lines 129 to 144 (Ctrl+G, type 129)

Say: Selection: sort by fitness, keep the top slice as parents, remember the champion.

```
129     WaveResults.Sort([](const FResult& A, const FResult& B) { return A.Fitness > B.Fitness; });
130
131     // The top slice become parents. At least two when we have them, so there is something to cross.
132     const int32 Wanted = FMath::CeilToInt(WaveResults.Num() * ParentFraction);
133     const int32 ParentCount = FMath::Clamp(FMath::Max(2, Wanted), 1, WaveResults.Num());
134     Parents.Reset();
135     for (int32 I = 0; I < ParentCount; ++I)
136     {
137         Parents.Add(WaveResults[I].Genome);
138     }
139
140     Champion = WaveResults[0].Genome;
141     Champion.bChampion = false;
142     ChampionFitness = WaveResults[0].Fitness;
143     bHasChampion = true;
144     bChampionIssued = false;
```

SHOW IN CODE

Source/ProceduralGenGade3B/EnemyEvolutionTests.cpp lines 132 to 133 (Ctrl+G, type 132)

Say: The test that proves selection works: the mean must rise by more than 0.08.

```
132     AddInfo(FString::Printf(TEXT("Mean Swiftness went from %.3f in generation 0 to %.3f in generation
8."), FirstMean, LastMean));
133     TestTrue(TEXT("selection pushes the population towards the trait that survives"), LastMean >
FirstMean + 0.08f);
```

CARD 8 OF 12

Plugging it into the game

0:20

clock 3:50 to 4:10

SAY (about 42 words, roughly 0:19 at a calm pace)

The Wave Manager asks for a genome at every spawn and reports every outcome. ApplyGenome turns the genes into health, speed, damage, loot, size and shader values. It runs before the elite modifier, so everything stacks. The seed comes from the map.

SHOW IN GAME

1. Stay in the game, a wave in progress.
2. Point at one fast small enemy and one big slow enemy walking side by side.

SHOW IN CODE

Source/ProceduralGenGade3B/WaveManager.cpp lines 348 to 358 (Ctrl+G, type 348)

Say: At every spawn: seed from the map, take the next genome, apply it to the enemy.

```
348         FEnemyGenome SpawnedGenome;
349         bool bGaveGenome = false;
350         if (bUseEvolvingEnemies && Evolution)
351         {
352             const ATDGameMode* GameMode = GetWorld()->GetAuthGameMode<ATDGameMode>();
353             const AProceduralTerrain* Terrain = GameMode ? GameMode->GetTerrain() : nullptr;
354             Evolution->EnsureSeeded(Terrain ? Terrain->Seed : 0);
355             SpawnedGenome = Evolution->NextGenome();
356             NewEnemy->ApplyGenome(SpawnedGenome);
357             bGaveGenome = true;
358         }
```

SHOW IN CODE

Source/ProceduralGenGade3B/Enemy.cpp lines 98 to 108 (Ctrl+G, type 98)

Say: Genes become health, damage, speed and loot. Danger pays: stronger genomes drop more.

```
 98      // ---- Stats. Every multiplier is 1.0 at a gene value of 0.5, so an average genome plays like the
old enemy. ----
 99      if (HealthComponent)
100      {
101          HealthComponent->MaxHealth *= Genome.HealthMultiplier();
102          HealthComponent->Heal(HealthComponent->MaxHealth); // fill up to the new maximum
103      }
104      AttackDamage *= Genome.DamageMultiplier();
105      MoveSpeed *= Genome.SpeedMultiplier();
106
107      // Danger pays: a stronger genome drops more loot.
108      ResourceReward = FMath::RoundToInt(ResourceReward * (0.7f + 0.5f * Genome.PowerRating()));
```

CARD 9 OF 12

More procedural enemies: boids and cellular poison

0:40

clock 4:10 to 4:50

SAY (about 85 words, roughly 0:39 at a calm pace)

Two more procedural enemies join from wave two. The swarm uses boids: each small enemy follows the lane but also keeps away from close neighbours, faces the way they face, and drifts to their centre, so the pack flows like a flock. The slime uses a cellular automaton. When it dies, or gets near a defender, it releases poison on a grid. Each infected cell can infect its neighbours, ages, and burns out. A defender standing in infected cells takes heavy damage and is destroyed.

SHOW IN GAME

1. Play to wave two: point at the pack of small yellow enemies moving together, splitting round each other.
2. Point at the green slime. Let it reach a defender or kill it: the green poison patch grows and fades.
3. Show a defender in the poison losing its health bar and disappearing.

If you have time, also point out:

- Both are seeded random processes, so every game plays out differently.
- Added to each wave's spawn queue in WaveManager so the wave only ends when they are gone.

SHOW IN CODE

Source/ProceduralGenGade3B/Enemy.cpp lines 558 to 594 (Ctrl+G, type 558)

Say: The three boid rules: separation, alignment and cohesion, added to the heading along the lane.

```
558      FVector Separation = FVector::ZeroVector; // steer away from flock mates that are too close
559      FVector Alignment = FVector::ZeroVector;  // steer the way the neighbours are facing
560      FVector Centre = FVector::ZeroVector;     // steer towards the middle of the neighbours
561      int32 Neighbours = 0;
562      for (TActorIterator<AEnemy> It(GetWorld()); It; ++It)
563      {
564          const AEnemy* Other = *It;
565          if (!Other || Other == this || Other->EnemyType != EEnemyType::Swarm || (Other-
>HealthComponent && Other->HealthComponent->IsDead()))
566          {
567              continue;
568          }
569          FVector Away = Location - Other->GetActorLocation();
570          Away.Z = 0.0f;
571          const float Dist = Away.Size();
572          if (Dist < 1.0f || Dist > 450.0f)
573          {
574              continue;
575          }
576          ++Neighbours;
577          Centre += Other->GetActorLocation();
578          Alignment += Other->GetActorForwardVector();
579          if (Dist < 140.0f)
580          {
581              Separation += Away / Dist * ((140.0f - Dist) / 140.0f);
582          }
583      }
584
585      FVector Steer = ToWaypoint.GetSafeNormal() + Separation * 2.2f;
586      if (Neighbours > 0)
587      {
588          FVector ToCentre = Centre / Neighbours - Location;
589          ToCentre.Z = 0.0f;
590          Alignment.Z = 0.0f;
591          Steer += ToCentre.GetSafeNormal() * 0.45f + Alignment.GetSafeNormal() * 0.6f;
     ... (continues in the file)
```

SHOW IN CODE

Source/ProceduralGenGade3B/PoisonCloud.cpp lines 85 to 128 (Ctrl+G, type 85)

Say: The cellular automaton step: infected cells age and burn out, clean cells catch poison from young neighbours.

```
 85      TArray<uint8> Next = Age;
 86      bool bAnyInfected = false;
 87
 88      for (int32 Y = 0; Y < GridSize; ++Y)
 89      {
 90          for (int32 X = 0; X < GridSize; ++X)
 91          {
 92              const uint8 Current = Age[Index(X, Y)];
 93              if (Current > 0)
 94              {
 95                  // Infected cells age, then burn out after CellLifetime steps.
 96                  Next[Index(X, Y)] = (Current + 1 > CellLifetime) ? 0 : Current + 1;
 97              }
 98              else
 99              {
100                  // A clean cell next to young infected cells may catch the poison.
101                  int32 YoungNeighbours = 0;
102                  for (int32 DY = -1; DY <= 1; ++DY)
103                  {
104                      for (int32 DX = -1; DX <= 1; ++DX)
105                      {
106                          const int32 NX = X + DX;
107                          const int32 NY = Y + DY;
108                          if ((DX == 0 && DY == 0) || NX < 0 || NY < 0 || NX >= GridSize || NY >=
GridSize)
109                          {
110                              continue;
111                          }
112                          const uint8 NeighbourAge = Age[Index(NX, NY)];
113                          if (NeighbourAge > 0 && NeighbourAge <= 3)
114                          {
115                              ++YoungNeighbours;
116                          }
117                      }
118                  }
     ... (continues in the file)
```

SHOW IN CODE

Source/ProceduralGenGade3B/PoisonCloud.cpp lines 152 to 175 (Ctrl+G, type 152)

Say: Any defender standing in an infected cell takes damage once per step, which is how poison destroys it.

```
152  void APoisonCloud::DamageDefenders()
153  {
154      for (TActorIterator<ADefender> It(GetWorld()); It; ++It)
155      {
156          ADefender* Defender = *It;
157          UHealthComponent* Health = Defender ? Defender->HealthComponent : nullptr;
158          if (!Health || Health->IsDead())
159          {
160              continue;
161          }
162          for (int32 Y = 0; Y < GridSize; ++Y)
163          {
164              for (int32 X = 0; X < GridSize; ++X)
165              {
166                  if (Age[Index(X, Y)] > 0 && FVector::Dist2D(CellWorldLocation(X, Y), Defender-
>GetActorLocation()) < CellSize * 0.75f)
167                  {
168                      Health->ApplyDamage(DamagePerStep, this); // one hit per step, however many cells
overlap
169                      X = GridSize;
170                      break;
171                  }
172              }
173          }
174      }
175  }
```

CARD 10 OF 12

Balance: waves react to upgrades

0:25

clock 4:50 to 5:15

SAY (about 47 words, roughly 0:21 at a calm pace)

To keep the game balanced with upgrades, the Wave Director measures the average upgrade tier when a wave ends, before the upgrades reset. It grows the next wave's threat budget by twelve percent per tier. The HUD tells the player when their upgrades made a wave bigger.

SHOW IN GAME

1. Buy the tower upgrade during a wave and survive until the wave ends.
2. On the next-wave forecast line point at 'threat N, +x% for upgrades'.

If you have time, also point out:

- The tower counts as one unit, so a tower-only defence is still measured.
- UpgradeBudgetPerTier = 0.12 is an editable property, no recompile needed to tune it.

SHOW IN CODE

Source/ProceduralGenGade3B/WaveDirector.cpp lines 187 to 189 (Ctrl+G, type 187)

Say: Measured at wave end, just before the game mode resets the upgrades.

```
 187      // Upgrades only last for one wave and the game mode resets them right after this function runs,
so
 188      // measure them now. The next wave is planned from this value.
 189      LastWaveUpgradeStrength = MeasureUpgradeStrength(World);
```

SHOW IN CODE

Source/ProceduralGenGade3B/WaveDirector.cpp lines 271 to 275 (Ctrl+G, type 271)

Say: The next wave's budget scales with how strongly the player upgraded.

```
 271      // A defence that used upgrades is stronger than its unit count suggests, so the wave grows to
match.
 272      // The difficulty rating already reacts to how well the player did, but it only moves a little
each wave;
 273      // this makes the budget respond to upgrades straight away.
 274      Plan.UpgradeBudgetFactor = 1.0f + UpgradeBudgetPerTier * Profile.UpgradeStrength;
 275      Plan.ThreatBudget *= Plan.UpgradeBudgetFactor;
```

CARD 11 OF 12

Challenges and how I solved them

0:20

clock 5:15 to 5:35

SAY (about 50 words, roughly 0:23 at a calm pace)

Three problems taught me the most. The damage flash reset enemy size after every hit, so I added ScaleBase. A settings screen rebound its buttons every frame, writing twelve thousand log lines in two minutes, so it now binds once. And the director had to measure upgrades before they reset.

SHOW IN GAME

1. Stay on the game or show the code editor.
2. Optional: show the log file size before/after to make the 12,450 lines concrete.

If you have time, also point out:

- Found by reading the log of a smoke test, not by guessing.
- Also replaced a missing Cascade asset with a Niagara system when the tower is destroyed.

SHOW IN CODE

Source/ProceduralGenGade3B/DamageFlashComponent.h lines 22 to 22 (Ctrl+G, type 22)

Say: The flash restores BaseScale after a hit, so a permanent size change has to update it too.

```
22      void ScaleBase(float Factor) { BaseScale *= Factor; }
```

SHOW IN CODE

Source/ProceduralGenGade3B/TDGameMode_UI.inl lines 25 to 31 (Ctrl+G, type 25)

Say: Bind once per widget instead of every frame: 12,450 log lines in two minutes became 1.

```
25          // Each settings widget only needs its handlers attached once. Without this check the Tick
26          // below re-attached every handler and logged a line on every frame (about 100 a second).
27          if (!Screen || BoundSettingsScreens.Contains(Screen))
28          {
29              continue;
30          }
31          BoundSettingsScreens.Add(Screen);
```

CARD 12 OF 12

Conclusion

0:15

clock 5:35 to 5:50

SAY (about 33 words, roughly 0:15 at a calm pace)

To summarise: two-tier upgrades, a vertex displacement shader, a post-process effect, and an evolving enemy ecosystem with flocking swarms and cellular poison, all tested and integrated into the game. Thank you for watching.

SHOW IN GAME

1. Return to a clear shot of the game with the tower in the middle.
2. Stop the recording after a one-second pause.

SHOW IN CODE

No code on this card. Keep the game on screen.

Questions you might be asked

Table 1: *Likely questions and answers*

Possible question	Suggested answer
Why a genetic algorithm and not a neural network?	Its rules stay readable, testable and tunable, which mattered for balance. Earlier tower defence work used both approaches (Hind and Harvey, 2024; Wattanapornprom, et al., 2024).
What stops the enemies becoming unbeatable?	Three limits: the shared biomass budget, a growth cap of 2.1, and gene limits. Evolution only redistributes a fixed budget.
How do you know it works?	Four automated tests, including a selection test where mean swiftness rises from 0.46 to 0.94, plus play-testing and log checks.
Which parts are yours and which are Belinda's?	Mine: terrain, the Wave Director, the second upgrade tier, the vertex shader, the post-process, upgrade scaling, Evolving Blight, the boid and poison enemies, the HUD cards, the model swap and the lighting. Belinda: the base upgrade system, Niagara effects, music, save/resume and the menu screens.
Why is the upgrade wave-scoped?	It keeps upgrades a per-wave decision and lets the director measure them. Prices and numbers are editable in one table.
What would you do with more time?	Save the evolving population, add a bestiary of past champions, and swap the static models for animated characters.